

Correctness Proof of Dijkstra's Shortest Path Algorithm

Setup and Definitions

Let $G = (V, E, w)$ be a weighted directed graph where $w : E \rightarrow \mathbb{R}_{\geq 0}$ assigns non-negative weights to edges. Fix a source vertex $s \in V$.

Definition 1. For a path $p = \langle v_0, v_1, \dots, v_k \rangle$, its *weight* is $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$. The *shortest-path distance* from s to v is

$$\delta(s, v) = \begin{cases} \min\{w(p) : p \text{ is a path from } s \text{ to } v\} & \text{if such a path exists,} \\ \infty & \text{otherwise.} \end{cases}$$

Algorithm

Algorithm 1 Dijkstra(G, s)

```
1:  $d[s] \leftarrow 0$ ;  $d[v] \leftarrow \infty$  for all  $v \neq s$ 
2:  $S \leftarrow \emptyset$  ▷ set of finalized vertices
3:  $Q \leftarrow V$  ▷ min-priority queue keyed by  $d[\cdot]$ 
4: while  $Q \neq \emptyset$  do
5:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
6:    $S \leftarrow S \cup \{u\}$ 
7:   for each neighbor  $v$  of  $u$  do
8:     if  $d[u] + w(u, v) < d[v]$  then
9:        $d[v] \leftarrow d[u] + w(u, v)$  ▷ relax edge  $(u, v)$ 
10:    end if
11:  end for
12: end while
```

Correctness

Theorem 1. When Dijkstra's algorithm terminates, $d[v] = \delta(s, v)$ for every $v \in V$.

We first establish the following loop invariant.

Lemma 1 (Loop Invariant). *At every point during the algorithm, for every vertex $u \in S$,*

$$d[u] = \delta(s, u).$$

Proof. We prove this by induction on $|S|$, the number of vertices that have been finalized.

Base case ($|S| = 0$). S is empty, so the invariant holds vacuously.

Inductive step. Assume the invariant holds for $|S| = k$. Let u be the next vertex extracted from Q , so $|S|$ grows to $k + 1$.

We must show $d[u] = \delta(s, u)$.

Since $d[u] \geq \delta(s, u)$ always (we never underestimate: $d[\cdot]$ records the weight of some explicit path, or ∞), it suffices to prove $d[u] \leq \delta(s, u)$, i.e. that no path to u is shorter than $d[u]$.

Suppose, for contradiction, that there exists a path p from s to u with $w(p) < d[u]$. Since $s \in S$ and $u \notin S$ (at the moment of extraction), path p must cross the cut $(S, V \setminus S)$ at some point. Let (x, y) be the *first* edge of p that crosses this cut, so $x \in S$ and $y \notin S$.

Let p_1 be the sub-path of p from s to x , and let p_2 be the remainder from y to u . Then:

$$w(p) = w(p_1) + w(x, y) + w(p_2).$$

By the inductive hypothesis, $d[x] = \delta(s, x)$. Since p_1 is a path from s to x , we have $\delta(s, x) \leq w(p_1)$, so:

$$d[y] \leq d[x] + w(x, y) \leq w(p_1) + w(x, y),$$

where the first inequality holds because the edge (x, y) was relaxed when x was finalized (by inductive hypothesis $d[x] = \delta(s, x)$, so the relaxation $d[y] \leq d[x] + w(x, y)$ was performed).

Because all edge weights are non-negative, $w(p_2) \geq 0$, so:

$$d[y] \leq w(p_1) + w(x, y) \leq w(p_1) + w(x, y) + w(p_2) = w(p) < d[u].$$

But $y \notin S$ and $u = \text{EXTRACT-MIN}(Q)$, meaning $d[u] \leq d[y]$ for all vertices still in Q . This gives $d[u] \leq d[y] < d[u]$, a contradiction.

Therefore no such path p exists, and $d[u] = \delta(s, u)$. The invariant is maintained. $\square$

Proof of Theorem. Every vertex $v \in V$ is eventually extracted from Q and added to S (the algorithm only terminates when $Q = \emptyset$). By the Lemma, upon extraction we have $d[v] = \delta(s, v)$, and $d[v]$ is never increased after that point. Hence, when the algorithm terminates, $d[v] = \delta(s, v)$ for all $v \in V$. $\square$

Why Non-Negative Weights Are Necessary

The argument above relies critically on $w(p_2) \geq 0$. If negative weights were permitted, a later edge on path p could make $w(p) < d[y]$ even though $d[y] \leq d[u]$, and the contradiction would not hold. Indeed, Dijkstra's algorithm can return incorrect results on graphs with negative-weight edges.

Complexity

With a binary min-heap, each `EXTRACT-MIN` costs $O(\log |V|)$ and each relaxation (decrease-key) costs $O(\log |V|)$. Over all $|V|$ extractions and at most $|E|$ relaxations:

$$T(|V|, |E|) = O((|V| + |E|) \log |V|).$$

With a Fibonacci heap this improves to $O(|V| \log |V| + |E|)$.